TikuOS Pool Allocator

Simple. Ubiquitous. Intelligence, Everywhere.

Ambuj Varshney
ambuj@tiku-os.org

March 11, 2026

Outline

1. Why a Pool Allocator?
2. Architecture & Data Structures
3. API Reference
4. Usage Examples
5. File Organization
6. API Summary
7. Test Suite
8. Summary

Arena vs. Pool: Complementary Strategies

Arena (bump-pointer):

- O(1) alloc, O(1) bulk free
- No individual free
- Ideal for shared-lifetime buffers
- Zero fragmentation

Pool (fixed-size block):

- O(1) alloc, O(1) individual free
- Embedded freelist — zero metadata
- Ideal for alloc/free of equal-sized objects
- Zero fragmentation

Arena Limitation

Cannot free individual objects. All allocations must be discarded together via `reset()`.

Pool Use Cases

Packet buffers, message slots, sensor sample records, timer descriptors — any pattern needing individual alloc/free of fixed-size objects.

Pool Control Block

kernel/memory/tiku_mem.h

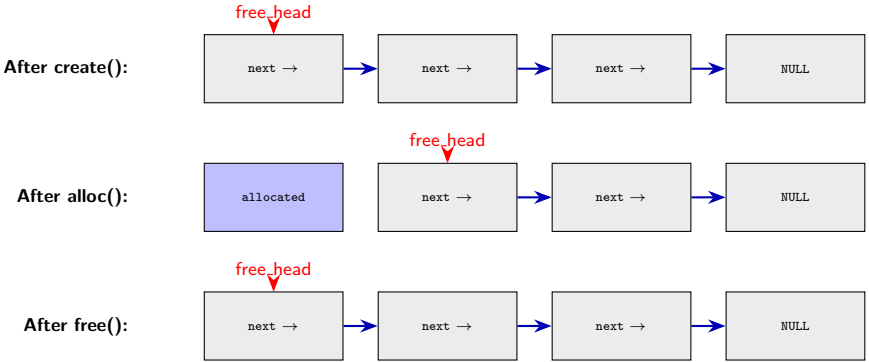
```
typedef struct {
    uint8_t *buf;
    tiku_mem_arch_size_t block_size;
    tiku_mem_arch_size_t block_count;
    void    *free_head;
    tiku_mem_arch_size_t used_count;
    tiku_mem_arch_size_t peak_count;
    uint8_t  id;
    uint8_t  active;
} tiku_pool_t;
```

Field	Purpose
buf	Caller-provided backing buffer
block_size	Aligned size of each block
block_count	Total number of blocks
free_head	Head of embedded freelist
used_count	Currently allocated blocks
peak_count	Lifetime high-water mark
id	Debug identifier (0–255)
active	Non-zero if initialized

Key Design

Like the arena, the pool **does not own** the buffer — the caller provides a statically allocated array. No dynamic allocation anywhere.

The Embedded Freelist



Zero Metadata Overhead

Each free block stores the next-pointer **inside its own memory** (first `sizeof(void *)` bytes). When allocated, those bytes are returned to the caller. No separate metadata structure needed.

Building the Freelist

kernel/memory/tiku_pool.c (private helper)

```
static void build_freelist(tiku_pool_t *pool)
{
    tiku_mem_arch_size_t i;
    uint8_t *block;
    void **next_ptr;

    for (i = 0; i < pool->block_count - 1U; i++) {
        block    = pool->buf + (i * pool->block_size);
        next_ptr = (void **)(void *)block;
        *next_ptr = block + pool->block_size;
    }

    /* Last block terminates the list */
    block    = pool->buf + ((pool->block_count - 1U) * pool->block_size);
    next_ptr = (void **)(void *)block;
    *next_ptr = NULL;

    pool->free_head = pool->buf;
}
```

Why `uint8_t *` Pointer Arithmetic

Struct padding and pointer size vary across platforms. By casting to `uint8_t *` and indexing by `i *`

tiku_pool_create()

```
tiku_mem_err_t tiku_pool_create(tiku_pool_t *pool, uint8_t *buf,
                                tiku_mem_arch_size_t block_size,
                                tiku_mem_arch_size_t block_count,
                                uint8_t id);
```

- Divides the buffer into `block_count` blocks of `block_size` bytes
- Rounds `block_size` up to `TIKU_MEM_ARCH_ALIGNMENT`
- Clamps to minimum of `sizeof(void *)` — room for the freelist pointer
- Chains all blocks into the embedded freelist
- Returns `TIKU_MEM_ERR_INVALID` if `pool` or `buf` is NULL, or `block_count` is 0

Usage:

```
static uint8_t pkt_buf[8 * 64];  /* 8 blocks of 64 bytes */
static tiku_pool_t pkt_pool;

tiku_pool_create(&pkt_pool, pkt_buf, 64, 8, 1);
```

tiku_pool_alloc() & tiku_pool_free()

Alloc — O(1), pop freelist

```c
void *tiku_pool_alloc(
    tiku_pool_t *pool)
{
    void *block;
    void **next_ptr;

    if (pool == NULL || !pool->active
        || pool->free_head == NULL)
        return NULL;

    /* Pop head */
    block = pool->free_head;
    next_ptr = (void **)(void *)block;
    pool->free_head = *next_ptr;

    pool->used_count++;
    if (pool->used_count >
        pool->peak_count)
        pool->peak_count =
            pool->used_count;

    return block;
}
```

Free — O(1), push freelist

```c
tiku_mem_err_t tiku_pool_free(
    tiku_pool_t *pool, void *ptr)
{
    uint8_t *block = (uint8_t *)ptr;
    tiku_mem_arch_size_t offset;
    void **next_ptr;

    /* Validate: in range? */
    if (block < pool->buf ||
        block >= pool->buf +
        (pool->block_count *
         pool->block_size))
        return TIKU_MEM_ERR_INVALID;

    /* Validate: aligned? */
    offset = (tiku_mem_arch_size_t)
        (block - pool->buf);
    if (offset % pool->block_size != 0)
        return TIKU_MEM_ERR_INVALID;

    /* Push onto head */
    next_ptr = (void **)(void *)block;
    *next_ptr = pool->free_head;
```

Free Pointer Validation

`tiku_pool_free()` performs two safety checks before accepting a pointer:

1. **Range check:** `ptr` must fall within `[buf, buf + block_count * block_size)`
2. **Alignment check:** `(ptr - buf) % block_size == 0`

This catches common embedded bugs:

- Freeing a pointer from a different allocator
- Freeing a stack or heap pointer
- Freeing at the wrong offset within a block

No MMU on MCUs

On a microcontroller with no MMU, a stray free corrupts the freelist silently, causing future allocs to return wild pointers. The validation catches this at the point of error.

Cost

Two comparisons + one modulo operation. Negligible compared to the consequences of silent freelist corruption.

Alignment & Minimum Block Size

kernel/memory/tiku_pool.c (private helpers)

```
static tiku_mem_arch_size_t align_up(tiku_mem_arch_size_t size)
{
    const tiku_mem_arch_size_t mask = TIKU_MEM_ARCH_ALIGNMENT - 1U;
    return (size + mask) & ~mask;
}

static tiku_mem_arch_size_t min_block_size(void)
{
    tiku_mem_arch_size_t ptr_size = (tiku_mem_arch_size_t)sizeof(void *);
    if (TIKU_MEM_ARCH_ALIGNMENT > ptr_size)
        ptr_size = TIKU_MEM_ARCH_ALIGNMENT;
    return align_up(ptr_size);
}
```

Why `min_block_size()`?

Each free block must hold at least one pointer for the embedded freelist. On MSP430, `sizeof(void *)` = 2. On a 64-bit host, it is 8. The minimum is `max(sizeof(void *),`

Platform	Alignment	Min Block
MSP430 (16-bit)	2	2
ARM Cortex-M	4	4
x86-64 host	4	8

tiku_pool_stats() & tiku_pool_reset()

Stats — O(1)

```
tiku_mem_err_t tiku_pool_stats(
    const tiku_pool_t *pool,
    tiku_mem_stats_t *stats)
{
    stats->total_bytes =
        pool->block_size *
        pool->block_count;
    stats->used_bytes =
        pool->block_size *
        pool->used_count;
    stats->peak_bytes =
        pool->block_size *
        pool->peak_count;
    stats->alloc_count =
        pool->used_count;
    return TIKU_MEM_OK;
}
```

Reset — O(n)

```
tiku_mem_err_t tiku_pool_reset(
    tiku_pool_t *pool)
{
    if (pool == NULL)
        return TIKU_MEM_ERR_INVALID;

    pool->used_count = 0;
    build_freelist(pool);

    return TIKU_MEM_OK;
}
```

Shared Stats Type

Pool uses the same `tiku_mem_stats_t` as the

Reset is O(n)

Unlike the arena (O(1) reset), the pool must rebuild the freelist by writing a next-pointer into every block — O(n) in `block_count`. In practice, pool sizes are small (8–32 blocks) so this is fast.

Debug Poisoning

kernel/memory/tiku_pool.c — inside tiku_pool_free()

```
#if TIKU_POOL_DEBUG
    /* Poison freed block to catch use-after-free during development */
    {
        tiku_mem_arch_size_t ptr_bytes = (tiku_mem_arch_size_t)sizeof(void *);
        tiku_mem_arch_size_t i;
        for (i = ptr_bytes; i < pool->block_size; i++) {
            block[i] = 0xDE;   /* recognizable pattern: "dead" */
        }
    }
#endif
```

How It Works

- First `sizeof(void *)` bytes are the freelist pointer
- Remaining bytes filled with `0xDE`
- Use-after-free reads garbage instead of stale data
- Controlled by `TIKU_POOL_DEBUG` macro

Enable at Compile Time

```
# Build with debug poisoning
gcc -DTIKU_POOL_DEBUG=1 ...

# Default: disabled (zero overhead)
```

Production

When to Choose: Arena vs. Pool

Use an **Arena** when...

- All allocations share a **common lifetime**
- You process a batch of data, then discard everything
- You never need to free individual objects

Example: Read 10 sensor values into a temporary buffer, format a packet, transmit, then `reset()` the arena. All data was temporary.

Use a **Pool** when...

- Objects have **independent lifetimes**
- Objects arrive and depart at unpredictable times
- All objects are the **same size**

Example: Radio packets arrive via interrupts. Each sits in a buffer until processed — some take 10 ms, others 500 ms. You need to free each one individually when done.

Together

Arena + Pool cover the two fundamental embedded allocation patterns. Neither uses `malloc`/`free`. Neither fragments.

Example 1: Producer–Consumer Packet Queue

```c
/* ISR produces packets, main loop consumes them at a different rate */
static uint8_t pkt_buf[8 * 48];     /* 8 slots of 48 bytes */
static tiku_pool_t pkt_pool;
static uint8_t *pending[8];         /* simple ring of pending packets */
static volatile uint8_t head, tail;

void radio_isr(void) {                          /* --- interrupt context --- */
    uint8_t *slot = tiku_pool_alloc(&pkt_pool);
    if (slot == NULL) { stats.dropped++; return; }  /* back-pressure */
    radio_read(slot, 48);
    pending[head++ & 7] = slot;                  /* enqueue */
}

void main_loop(void) {                          /* --- thread context --- */
    while (head != tail) {
        uint8_t *pkt = pending[tail++ & 7];     /* dequeue */
        process(pkt);
        tiku_pool_free(&pkt_pool, pkt);         /* return slot -- O(1) */
    }
}
```

Why a Pool Here?

Packets arrive via ISR and are consumed later. Each packet has an **independent lifetime** — arena's bulk reset would discard in-flight packets. The pool lets each slot be freed individually.

Example 2: Timer Descriptor Pool

```c
/* Fixed-size timer descriptors: alloc when started, free when expired */
typedef struct {
    uint16_t  ticks_remaining;
    void     (*callback)(void *);
    void      *ctx;
} timer_desc_t;

static uint8_t tmr_buf[4 * sizeof(timer_desc_t)];
static tiku_pool_t tmr_pool;

timer_desc_t *timer_start(uint16_t ticks, void (*cb)(void *), void *ctx) {
    timer_desc_t *t = tiku_pool_alloc(&tmr_pool);
    if (t == NULL) return NULL;    /* no free timer slots */
    t->ticks_remaining = ticks;
    t->callback = cb;
    t->ctx = ctx;
    return t;
}

void timer_tick(void) {
    /* walk active timers; when one expires: */
    t->callback(t->ctx);
    tiku_pool_free(&tmr_pool, t);  /* reclaim the descriptor */
}
```

Example 3: Capacity Planning with Stats

```c
static uint8_t pkt_buf[16 * 48];
static tiku_pool_t pkt_pool;

void net_init(void) {
    tiku_pool_create(&pkt_pool, pkt_buf, 48, 16, 1);
}

void health_report(void) {
    tiku_mem_stats_t s;
    tiku_pool_stats(&pkt_pool, &s);

    TIKU_PRINTF("Pool: %u/%u bytes used (%u blocks)\n",
                s.used_bytes, s.total_bytes, s.alloc_count);
    TIKU_PRINTF("Peak: %u bytes (lifetime max)\n", s.peak_bytes);

    /* Right-sizing: if peak is always < 50% of total,
       the pool is over-provisioned -- shrink and save RAM */
    if (s.peak_bytes < s.total_bytes / 2)
        TIKU_PRINTF("  -> pool may be over-provisioned\n");
}
```

Peak Never Decreases

peak_bytes records the maximum concurrent usage across the **entire lifetime** of the pool, even across resets. Use it to right-size buffers: if peak is always 3 out of 16 slots, you only need 4 slots (with one

Example 4: Graceful Back-Pressure

```c
/* When the pool is exhausted, handle it gracefully instead of crashing */

void on_packet_received(void) {
    uint8_t *pkt = tiku_pool_alloc(&pkt_pool);

    if (pkt == NULL) {
        /* Pool exhausted -- apply back-pressure */
        radio_send_nack();      /* tell sender to slow down */
        stats.rx_dropped++;     /* count for diagnostics */
        return;                 /* do NOT process this packet */
    }

    radio_read(pkt, 48);
    enqueue_for_processing(pkt);
}

void processing_done(uint8_t *pkt) {
    tiku_pool_free(&pkt_pool, pkt);  /* free returns ERR if bad ptr */
}
```

No Crash, No Corruption

`tiku_pool_alloc()` returns NULL when empty — never crashes. `tiku_pool_free()` validates the pointer — never corrupts the freelist from a stray pointer. Both are O(1).

Example 5: Multi-Pool for Different Object Sizes

```c
/* Two independent pools for different subsystems */
static uint8_t msg_buf[16 * 32];   /* 16 message slots, 32 B each */
static uint8_t sen_buf[8 * 12];    /* 8 sensor records, 12 B each  */
static tiku_pool_t msg_pool, sen_pool;

void init(void) {
    tiku_pool_create(&msg_pool, msg_buf, 32, 16, 1);
    tiku_pool_create(&sen_pool, sen_buf, 12,  8, 2);
}

void on_sensor_sample(void) {
    uint8_t *rec = tiku_pool_alloc(&sen_pool);
    if (rec == NULL) return;
    read_sensor(rec, 12);
    log_to_flash(rec);
    tiku_pool_free(&sen_pool, rec);   /* return individually */
}

void on_message(void) {
    uint8_t *msg = tiku_pool_alloc(&msg_pool);
    if (msg == NULL) return;
    parse_message(msg, 32);
    tiku_pool_free(&msg_pool, msg);
}
```

Pool Allocator Files

Layer	File	Role
Kernel (portable)	`kernel/memory/tiku_mem.h`	API, types, `tiku_pool_t`
	`kernel/memory/tiku_pool.c`	Pool logic, freelist, validation
Arch (MSP430)	`arch/msp430/tiku_mem_arch.h`	Alignment, size type
	`hal/tiku_mem_hal.h`	HAL routing header
Tests	`tests/memory/test_mem_pool.c`	14 pool test cases
	`tests/memory/test_mem_common.c`	Host stubs + test main
	`tests/tiku_test_config.h`	Per-test enable flags

Portability

The pool allocator is entirely portable. It depends only on `TIKU_MEM_ARCH_ALIGNMENT` and `tiku_mem_arch_size_t` from the HAL. The same `tiku_pool.c` compiles on MSP430 and on a 64-bit host without changes.

API at a Glance

Function	Returns	Complexity	Description
`tiku_pool_create`	`err_t`	O(n)	Init pool, build freelist
`tiku_pool_alloc`	`void*`	O(1)	Pop freelist head
`tiku_pool_free`	`err_t`	O(1)	Validate & push to freelist
`tiku_pool_stats`	`err_t`	O(1)	Snapshot of pool state
`tiku_pool_reset`	`err_t`	O(n)	Rebuild freelist, preserve peak

Error Handling

- NULL checks on every public function
- `alloc()` returns NULL on failure
- `free()` validates range + alignment
- No assertions — safe for production

Design Guarantees

- No dynamic allocation anywhere
- O(1) individual alloc and free
- Zero fragmentation (all blocks equal)
- Peak survives resets and frees
- Optional debug poisoning (0xDE)

Test Configuration

tests/tiku_test_config.h

```
/* Pool allocator test flags */
#define TEST_POOL_CREATE        1
#define TEST_POOL_ALLOC_FREE    1
#define TEST_POOL_EXHAUSTION    1
#define TEST_POOL_FREE_RANGE    1
#define TEST_POOL_FREE_ALIGN    1
#define TEST_POOL_REALLOC       1
#define TEST_POOL_PEAK          1
#define TEST_POOL_RESET         1
#define TEST_POOL_INVALID       1
#define TEST_POOL_TWO_POOLS     1
#define TEST_POOL_BLOCK_ALIGN   1
#define TEST_POOL_STATS         1
#define TEST_POOL_POISON        1
#define TEST_POOL_WITHIN_BUF    1
```

On-target:

```
# Enable flags in tiku_test_config.h
# then build and flash
make flash MCU=msp430fr5969
make monitor
```

Host-native (no hardware):

```
gcc -std=c99 -Wall -Wextra -I. \
  tests/memory/test_mem_common.c \
  tests/memory/test_mem_pool.c \
  kernel/memory/tiku_pool.c \
  kernel/memory/tiku_mem.c \
  -o test_pool && ./test_pool
```

Host mode provides portable stubs for arch functions — no MSP430 toolchain required.

Test Cases Overview

#	Test Flag	What It Verifies
32	TEST_POOL_CREATE	Pool active; ID set; initial stats zero; freelist built
33	TEST_POOL_ALLOC_FREE	Non-NULL pointers; distinct blocks; non-overlap; free OK
34	TEST_POOL_EXHAUSTION	NULL after all blocks used; alloc after free succeeds
35	TEST_POOL_FREE_RANGE	Out-of-range pointer rejected (different buf, past end)
36	TEST_POOL_FREE_ALIGN	Misaligned pointer rejected (offset 1, block+1)
37	TEST_POOL_REALLOC	LIFO: freed blocks returned in reverse order
38	TEST_POOL_PEAK	Peak grows; preserved after free; never decreases
39	TEST_POOL_RESET	Freelist rebuilt; used 0; peak preserved; base realloc
40	TEST_POOL_INVALID	NULL pool/buf/ptr and zero count all rejected
41	TEST_POOL_TWO_POOLS	Independent pools; isolated reset and free
42	TEST_POOL_BLOCK_ALIGN	Min size enforced; odd sizes rounded to alignment
43	TEST_POOL_STATS	Stats mapping: bytes = block_size $\times$ count
44	TEST_POOL_POISON	Freed bytes filled with 0xDE (TIKU_POOL_DEBUG=1)
45	TEST_POOL_WITHIN_BUF	All allocated blocks fall within buffer bounds

Test Deep Dive: Exhaustion & LIFO Realloc

Pool exhaustion test

```c
uint8_t buf[4 * sizeof(void *)];
tiku_pool_t pool;
void *blocks[4];

tiku_pool_create(&pool, buf,
    sizeof(void *), 4, 1);

/* Allocate all 4 blocks */
for (i = 0; i < 4; i++)
    blocks[i] = tiku_pool_alloc(&pool);

/* Pool empty -> NULL */
void *p = tiku_pool_alloc(&pool);
TEST_ASSERT(p == NULL);

/* Free one -> alloc succeeds */
tiku_pool_free(&pool, blocks[0]);
p = tiku_pool_alloc(&pool);
TEST_ASSERT(p != NULL);
```

LIFO freelist test

```c
tiku_pool_create(&pool, buf, 8, 4, 1);

void *p1 = tiku_pool_alloc(&pool);
void *p2 = tiku_pool_alloc(&pool);

/* Free p2, then p1 */
tiku_pool_free(&pool, p2);
tiku_pool_free(&pool, p1);

/* LIFO: next alloc returns p1 */
void *p3 = tiku_pool_alloc(&pool);
TEST_ASSERT(p3 == p1);

/* Then p2 */
p3 = tiku_pool_alloc(&pool);
TEST_ASSERT(p3 == p2);
```

LIFO Order

Free pushes onto the head. The last freed block is the first returned by the next alloc.

Test Deep Dive: Free Validation & Debug Poisoning

Free validation test

```c
uint8_t buf[32];
uint8_t other_buf[8];
tiku_pool_t pool;

tiku_pool_create(&pool, buf, 8, 4, 1);

/* Out-of-range pointer */
tiku_mem_err_t err;
err = tiku_pool_free(&pool, other_buf);
TEST_ASSERT(err ==
    TIKU_MEM_ERR_INVALID);

/* Past buffer end */
err = tiku_pool_free(&pool, buf + 32);
TEST_ASSERT(err ==
    TIKU_MEM_ERR_INVALID);

/* Misaligned: offset 1 */
err = tiku_pool_free(&pool, buf + 1);
TEST_ASSERT(err ==
    TIKU_MEM_ERR_INVALID);
```

Debug poisoning test

```c
/* Compile with -DTIKU_POOL_DEBUG=1 */
tiku_pool_create(&pool, buf, 16, 4, 1);

void *p = tiku_pool_alloc(&pool);
memset(p, 0xAA, pool.block_size);

/* Free the block */
tiku_pool_free(&pool, p);

/* Verify: bytes after freelist ptr
   are poisoned with 0xDE */
uint8_t *block = (uint8_t *)p;
for (i = sizeof(void *);
     i < pool.block_size; i++) {
    TEST_ASSERT(block[i] == 0xDE);
}
```

Use-After-Free

Reads stale block data? 0xDE makes the bug immediately visible in hex dumps.

Summary

Pool Allocator Design

- Fixed-size block allocator
- O(1) alloc, O(1) individual free
- Embedded freelist — zero metadata
- Zero fragmentation
- Static buffers only
- Platform-aware alignment
- Lifetime peak tracking
- Free-pointer validation
- Optional debug poisoning (0xDE)

Test Coverage

- 14 comprehensive test cases
- Runs on-target or host-native
- Per-test enable flags
- Covers: create, alloc, free, exhaustion, validation, LIFO, peak, reset, invalid inputs, multi-pool, alignment, stats, poisoning, bounds

Complements the Arena

Use the arena for shared-lifetime bulk allocations. Use the pool for individual alloc/free of equal-sized objects. Together they cover embedded allocation needs without malloc/free.

Thank You

TikuOS Pool Allocator

Fixed-Size Block Allocator for Physical AI Endpoints

Repository:	`https://github.com/tikuos/tikuOS`
License:	Apache 2.0
API Header:	`kernel/memory/tiku_mem.h`
Implementation:	`kernel/memory/tiku_pool.c`
Tests:	`tests/memory/test_mem_pool.c`